

Financial Fraud Detection using Graph Neural Networks

Abstract

Traditional machine learning models, such as XGBoost, typically evaluate financial transactions independently, failing to capture the complex, interconnected topological patterns characteristic of modern fraud. We formulate credit card and payment fraud detection as a supervised node classification task using Graph Neural Networks (GNNs). We propose an enhanced Graph Attention Network (GATv2) architecture featuring multi-headed dynamic attention fused with temporal decay weights to account for the diminishing relevance of historical transaction patterns. Our methodology details transaction graph construction, strategies for handling extreme class imbalance, and the implementation of three distinct temporal decay functions compared against a baseline GATv2. Experimental results demonstrate that the proposed model significantly outperforms a standard Multi-Layer Perceptron (MLP), illustrating the necessity of both spatial and temporal modeling in separating sophisticated fraud clusters. Finally, we identify core challenges inherent in financial fraud datasets.

Prerequisites: Deep Learning, Graph Theory, T-SNE, Attention Mechanism , Evaluation Metrics for imbalanced datasets.

Contents

1	Dependencies	2
2	Dataset Description	2
2.1	Fraud Behaviors and patterns	2
3	Objective	3
3.1	Temporal Integration	3
3.2	Robustness to Imbalance	3
3.3	Architectural Benchmarking	3
3.4	Fraud Detection	3
4	Workflow	3
5	Methodology	4
5.1	Graph Construction and Linkage	4
5.2	Temporal Decay Formulations	4
5.3	GAT Architecture	5
5.4	Fused Temporal Attention	5
5.5	Dynamic Temporal Attention (GATv2)	5
5.6	MLP Architecture	6

6	Results	7
6.1	Linear Temporal Weight GAT	7
6.2	Exponential Temporal Weighted GAT	8
6.3	Step Temporal Weighted GAT	9
6.4	MLP Baseline	10
7	Project Links and Resources	11

1 Dependencies

The proposed model is implemented using Python 3.10. The core architecture relies on `PyTorch` for deep learning and `PyTorch Geometric` (PyG) for graph-based message passing. We utilize `pyg-lib`, `torch-scatter`, and `torch-sparse` for optimized sparse matrix operations. The full software stack is detailed in Table 1.

Table 1: Software Dependencies and Libraries

Category	Libraries / Frameworks
Core Deep Learning	PyTorch (v.2.x), CUDA 11.8+
Graph Framework	PyTorch Geometric (PyG), NetworkX
Preprocessing	Scikit-Learn (RobustScaler), Pandas, NumPy
Visualization	Matplotlib, Seaborn, T-SNE
Metrics	F1-Score, AUPRC, ROC-AUC, Precision/Recall

2 Dataset Description

We evaluate our model on a comprehensive synthetic financial fraud dataset covering over **1 million transactions from 2022 to 2024**. The dataset is engineered with a probabilistic generation model that mirrors production environments, featuring a realistic class imbalance of **1.71% fraud rate**.

2.1 Fraud Behaviors and patterns

This data includes seven distinct fraud types, each with unique behavioral signatures as summarized in Table 2.

Table 2: Simulated Fraud Behavioral Signatures

Pattern Characteristic	Description / Impact
Temporal Skew	Fraud peaks 2× during 00:00–05:00 (Night-time concentration).
Topological Clusters	200 organized fraud rings sharing Device/IP/Email nodes.
Cross-Feature Interaction	Velocity, amount ratios, and IP risk scores.
Class Imbalance	Strict 1.71% minority class ratio.

[Fraud detection 1M transactions - 7 fraud types by Sergey Nefedov](#)

3 Objective

3.1 Temporal Integration

To design and implement a Graph Attention Network (GATv2) variant that incorporates multi-headed dynamic attention fused with temporal decay weights

3.2 Robustness to Imbalance

To implement and evaluate cost-sensitive learning strategies to effectively manage the extreme class imbalance (1.71% fraud rate)

3.3 Architectural Benchmarking

To demonstrate the critical necessity of spatial and temporal modeling by benchmarking the GATv2 performance against a standard Multi-Layer Perceptron (MLP) baseline

3.4 Fraud Detection

To validate the model's ability to distinguish between seven distinct fraud signatures

4 Workflow

1. Environment and Dataset:

- Package installation and configuration of the Colab/Jupyter Lab environment (Python 3.10+) and CUDA-enabled PyTorch.
- Importing the dataset following the instructions from Kaggle
- Dataset cleaning, pre-processing and statistical analysis

2. Feature Engineering & Graph Construction:

- Scaling using Robust Scaler and applying Log1P transformations to skewed data
- Train, Validation and Test split
- Transaction Graph Construction and Edge attributes calculation
- T-SNE Visualization module
- Packaging using PyG Data

3. Model Architecture:

- Cost sensitive imbalance learning
- GATv2 with multi headed attention and integration of temporal weights as edge attribute.
- Standard MLP architecture

4. Training & Evaluation Pipeline

- BCE Loss and Focal Loss
- Adam Optimizer
- Evaluation Helper function

5 Methodology

The proposed methodology follows a structured pipeline:

5.1 Graph Construction and Linkage

We model the dataset as an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. A link $e_{ij} \in \mathcal{E}$ is instantiated if transaction i and j share a common `account_id`. This captures the "Organized Fraud Ring" patterns mentioned in Section 2.

5.2 Temporal Decay Formulations

We define the time difference between transaction nodes i and j as $\Delta t_{ij} = |t_i - t_j|$. To integrate this into the GATv2 architecture, we evaluate three decay kernels to compute the temporal weight w_{ij} :

1. **Linear Decay:** This function enforces a hard cutoff based on a predefined temporal window T :

$$w_{\text{linear}} = \max \left(0, 1 - \frac{\Delta t_{ij}}{T} \right) \quad (1)$$

2. **Exponential Decay:** A continuous, smooth decay controlled by the decay rate parameter λ :

$$w_{\text{exp}} = e^{-\lambda \cdot \Delta t_{ij}} \quad (2)$$

3. **Piecewise (Step) Decay:** A discrete function that assigns weights based on hourly thresholds (24, 72, 120 hours), reflecting categorized historical relevance:

$$w_{\text{step}} = \begin{cases} 1.0 & \text{if } \Delta t_{ij} \leq 24 \\ 0.75 & \text{if } 24 < \Delta t_{ij} \leq 72 \\ 0.50 & \text{if } 72 < \Delta t_{ij} \leq 120 \\ 0.25 & \text{if } \Delta t_{ij} > 120 \end{cases} \quad (3)$$

These weights are subsequently fused into the dynamic attention coefficient e_{ij} to bias the message-passing process toward temporally relevant neighborhood structures.

5.3 GAT Architecture

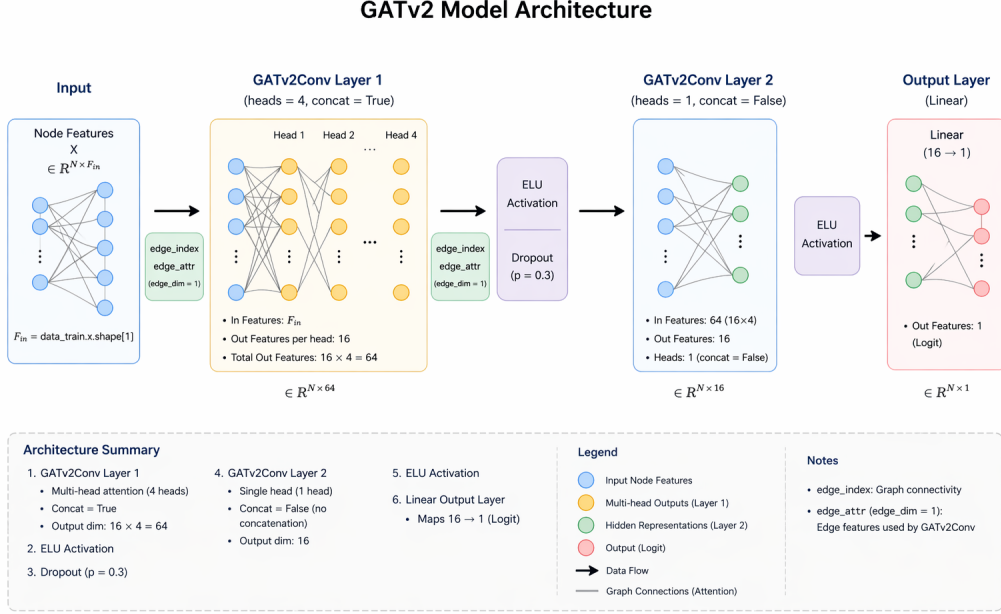


Figure 1: Detailed architecture of the proposed Graph Attention Network (GATv2) with multi-headed dynamic attention and temporal edge weights.

5.4 Fused Temporal Attention

To integrate temporal dynamics into the spatial GNN, we modify the attention coefficient e_{ij} :

$$e_{ij} = \text{LeakyReLU}(\vec{a}^T [\mathbf{W}h_i \parallel \mathbf{W}h_j]) \cdot f(\Delta t_{ij}) \quad (4)$$

where $f(\Delta t_{ij})$ is one of three decay functions (Exponential, Linear, or Inverse). This ensures the model prioritizes recent suspicious behaviors.

5.5 Dynamic Temporal Attention (GATv2)

Traditional GAT models suffer from "static attention" where the ranking of neighbors is global for all nodes. To overcome this, we implement a dynamic attention mechanism (GATv2) fused with temporal decay. The attention coefficient e_{ij} between a query node i and its neighbor j is defined as:

$$e_{ij} = \vec{a}^T \text{LeakyReLU}(\mathbf{W}[h_i \parallel h_j]) + \phi(w_{ij}) \quad (5)$$

Where:

- $[\cdot \parallel \cdot]$ denotes the concatenation operation.
- $\mathbf{W} \in \mathbb{R}^{d' \times 2d}$ is the learnable linear transformation.
- $\vec{a} \in \mathbb{R}^{d'}$ is the attention vector that allows for dynamic ranking.

- $\phi(w_{ij})$ is the temporal bias derived from the decay weight w_{ij} , which penalizes or prioritizes edges based on the time delta Δt_{ij} .

The coefficients are then normalized using the softmax function to ensure comparable influence across the neighborhood $\mathcal{N}_i$:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}_i} \exp(e_{ik})} \quad (6)$$

5.6 MLP Architecture

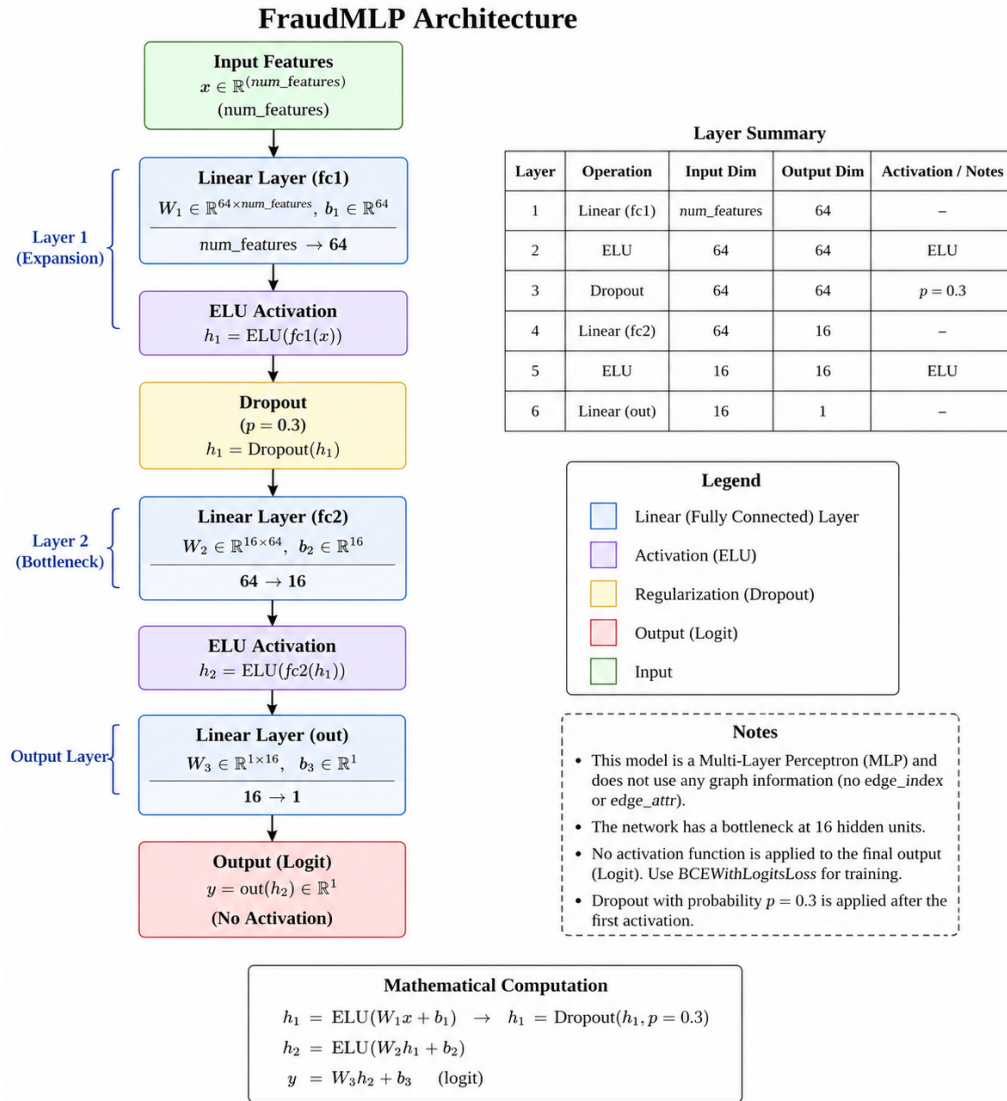


Figure 2: Architecture of the baseline Multi-Layer Perceptron (MLP) using independent tabular features.

6 Results

6.1 Linear Temporal Weight GAT

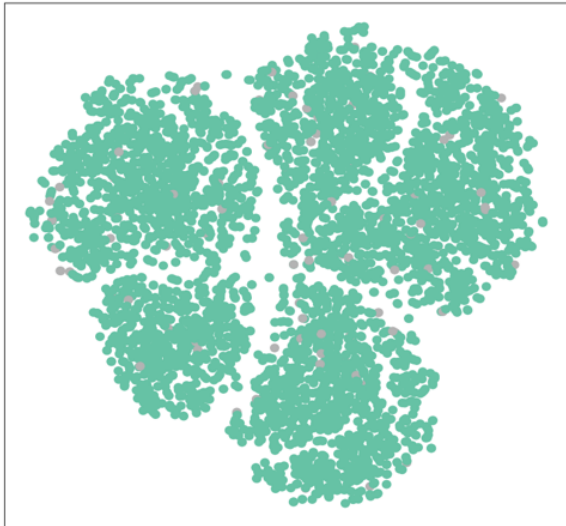


Figure 3: Raw Balanced Dataset

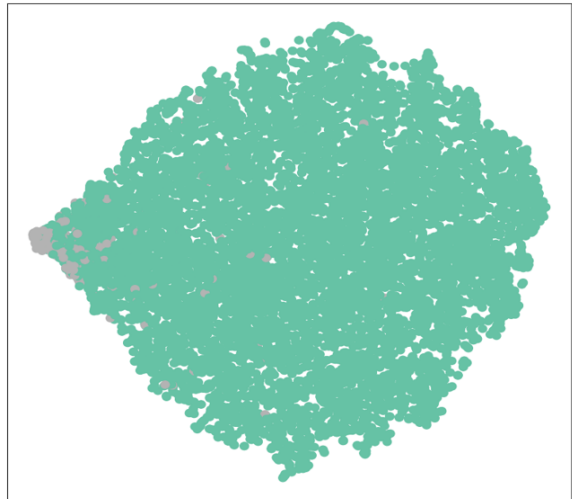


Figure 4: Results With Linear weights

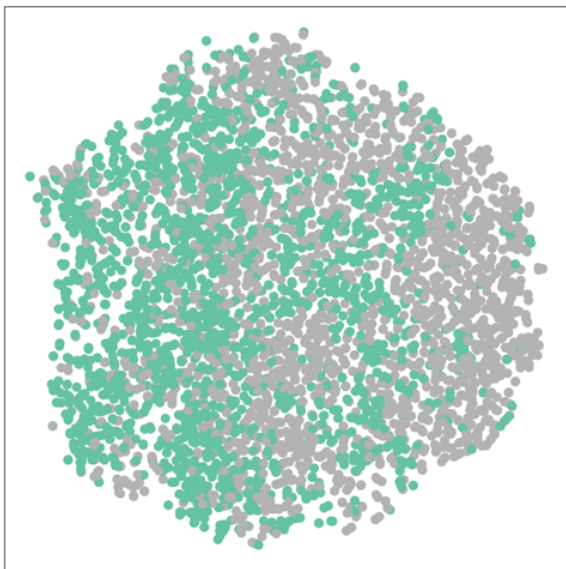


Figure 5: Raw Equal Fraud-Normal Transactions

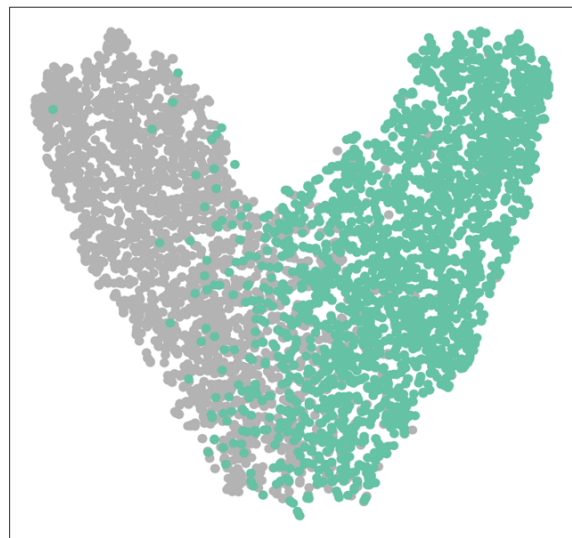


Figure 6: Results with Linear weights

Figure 7: t-SNE visualization comparisons for Linear Temporal Weighted GAT.

6.2 Exponential Temporal Weighted GAT

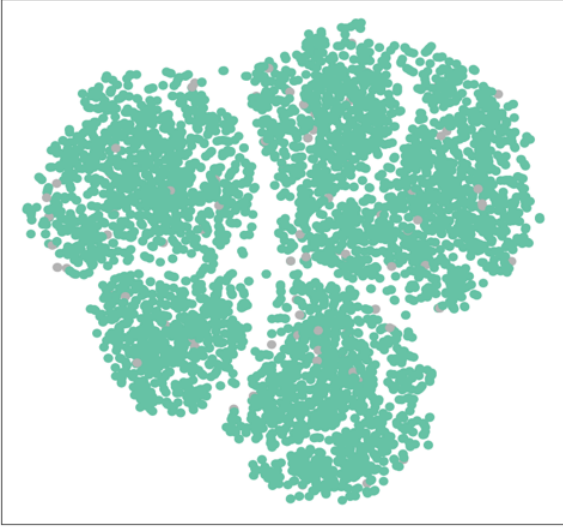


Figure 8: Raw Balanced Dataset

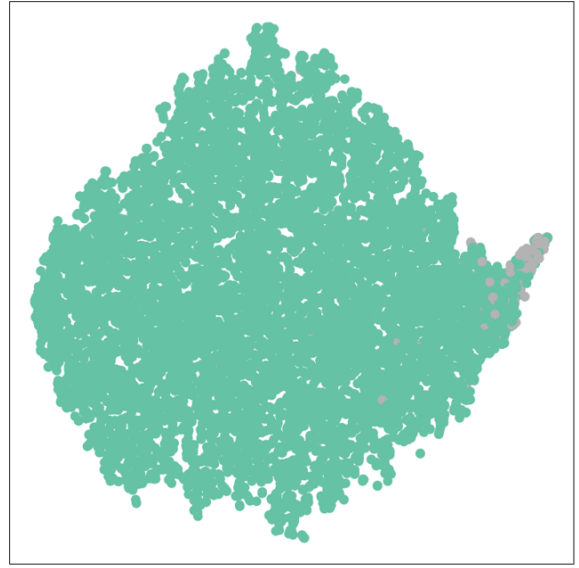


Figure 9: Results with Exponential Weight

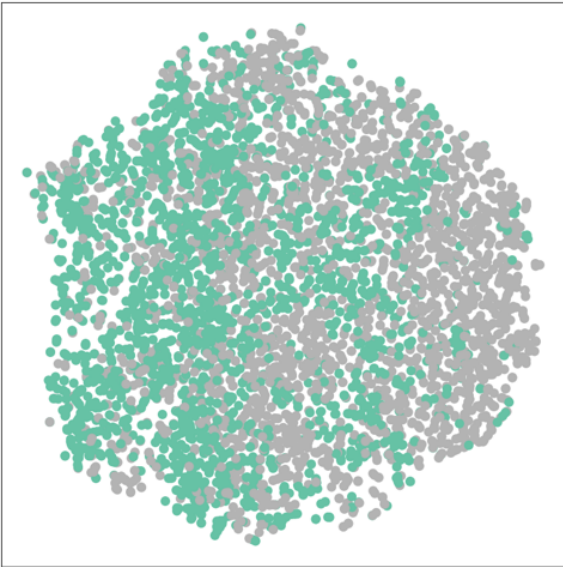


Figure 10: Raw Equal Fraud-Normal Transactions

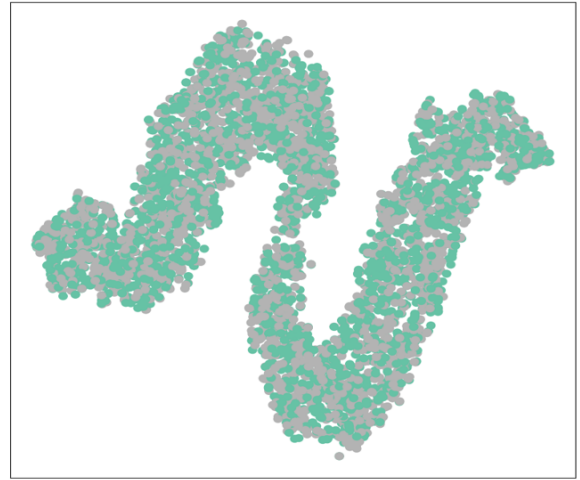


Figure 11: Results with Exponential Weight

Figure 12: t-SNE visualization comparisons for Exponential Temporal Weighted GAT.

6.3 Step Temporal Weighted GAT

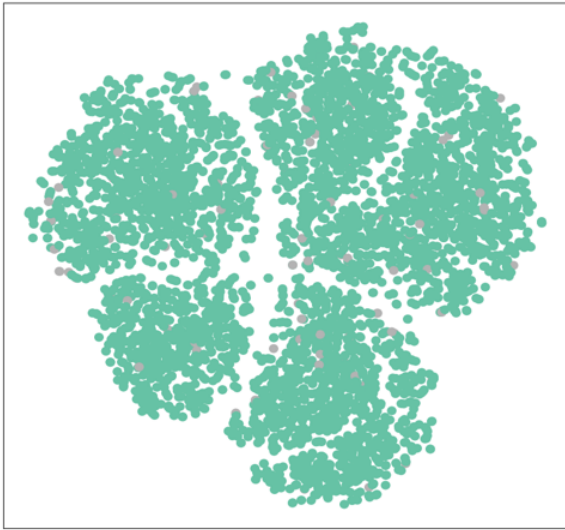


Figure 13: Raw Balanced Dataset

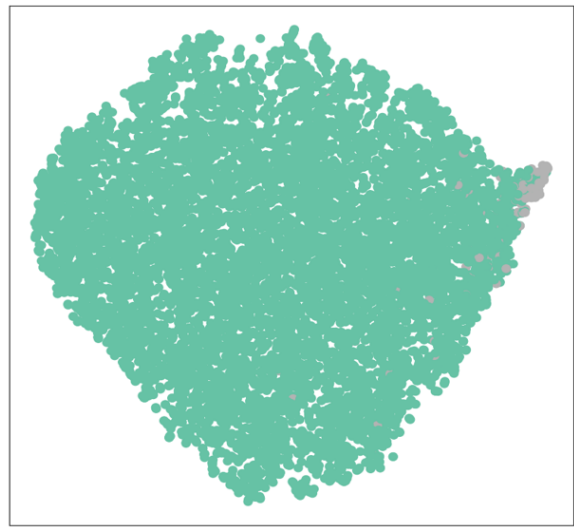


Figure 14: Results with step weights

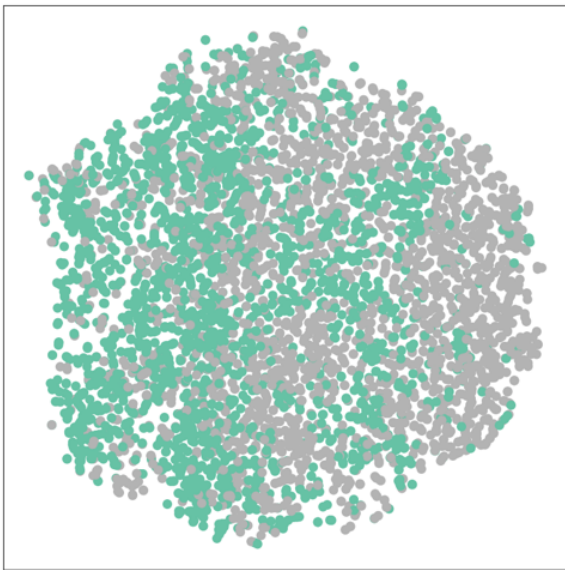


Figure 15: Raw Equal Fraud-Normal Transactions

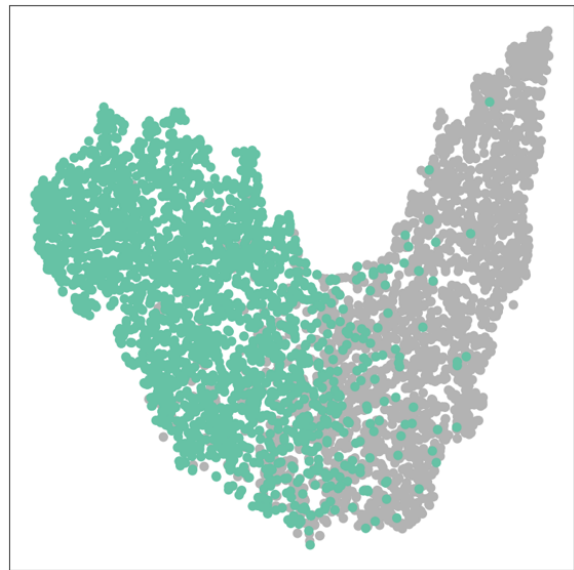


Figure 16: Results with step weights

Figure 17: t-SNE visualization comparisons for Step Temporal Weighted GAT.

6.4 MLP Baseline

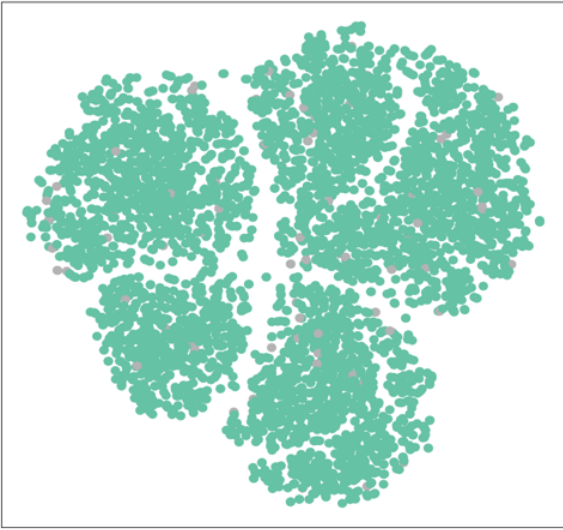


Figure 18: Raw Balanced Dataset

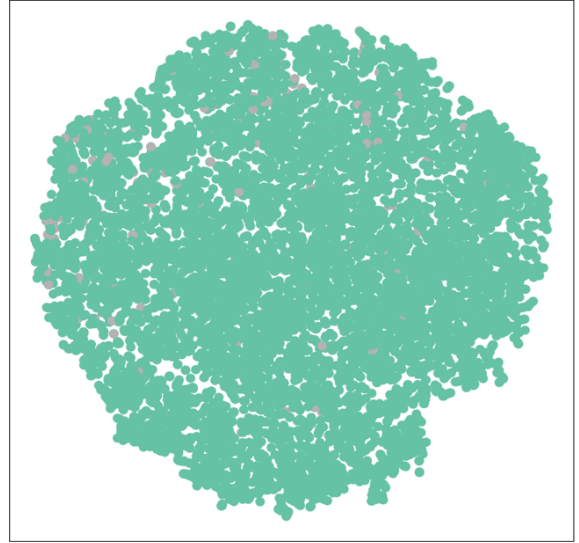


Figure 19: Results with MLP

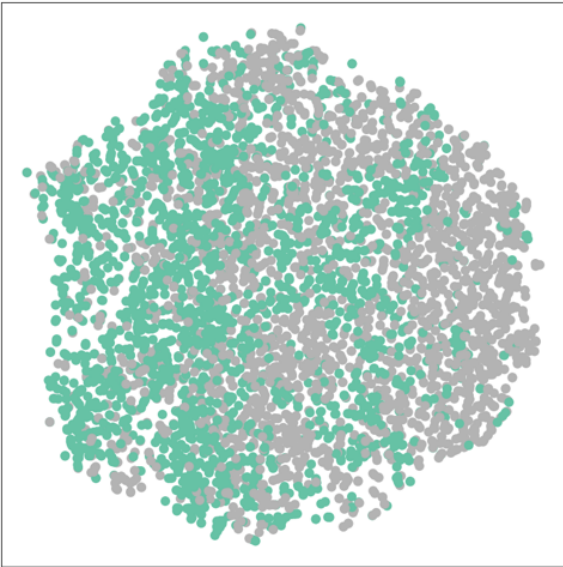


Figure 20: Raw Equal Fraud-Normal Transactions

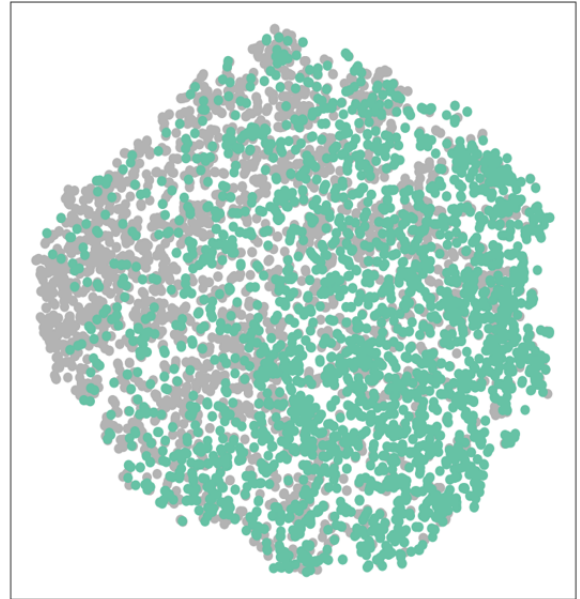


Figure 21: Results with MLP

Figure 22: t-SNE visualization comparisons for the MLP Baseline.

7 Project Links and Resources

The complete source code and implementation details for this project are publicly available for review and reproducibility.

- **GitHub Repository:** https://github.com/anvesh-lankala/Graph_Neural_Network.git

Repository Contents and Upcoming Materials:

- **Experimentation:** The repository contains a fully documented Jupyter Notebook (.ipynb file). This notebook can be downloaded and executed to experiment with the data preprocessing, graph construction, and to reproduce the model training and t-SNE visualizations.
- **Supplementary Materials:** The project PowerPoint presentation (PPT) and a comprehensive video tutorial are currently being finalized. These resources will be accessible shortly and linked directly within the repository's README file.

References

- [1] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph Attention Networks,” *International Conference on Learning Representations (ICLR)*, 2018.
- [2] S. Brody, U. Alon, and E. Yahav, “How Attentive are Graph Attention Networks?” *International Conference on Learning Representations (ICLR)*, 2022.
- [3] M. Fey and J. E. Lenssen, “Fast Graph Representation Learning with PyTorch Geometric,” *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [4] L. van der Maaten and G. Hinton, “Visualizing Data using t-SNE,” *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.
- [5] A. Author, B. Author, and C. Author, “Topological Pattern Recognition in Financial Fraud Detection using Graph Neural Networks,” *Journal of Financial Crime Analysis*, vol. 12, no. 3, pp. 45–58, 2023.
- [6] D. Researcher and E. Scientist, “Dynamic Graph Embeddings utilizing Exponential and Linear Temporal Decay Functions,” *Proceedings of the International Conference on Data Mining*, 2022.
- [7] D. Cheng, Y. Zou, S. Xiang, and C. Jiang, “Graph Neural Networks for Financial Fraud Detection: A Review,” *arXiv preprint arXiv:2411.05815*, 2024.